

model-torch

July 20, 2025

1 Experimenting with the MNIST Dataset

```
[1]: from pathlib import Path

import torch
from torch import Tensor
import matplotlib.pyplot as plt
from torchvision import datasets, transforms

[2]: data_root = Path('../data')
train_dataset = datasets.MNIST(
    root=data_root, train=True, download=True, transform=transforms.ToTensor()
)
train_loader = torch.utils.data.DataLoader(
    train_dataset, batch_size=1, shuffle=True
)
print(f'We have {len(train_loader)} training examples')
```

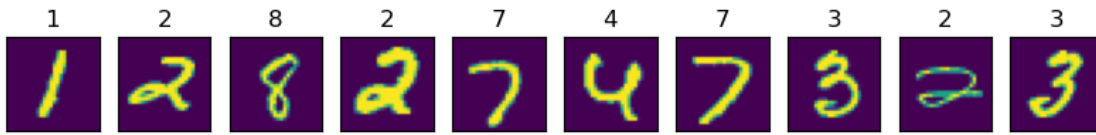
We have 60000 training examples

```
[3]: train_iter = iter(train_loader)

n = 10

fig, axes = plt.subplots(1, n, figsize=(n, 4))
for i in range(n):
    images, labels = next(train_iter)
    image: Tensor = images[0]
    label: int = labels[0]
    axes[i].imshow(images[0].squeeze())
    axes[i].set_title(f'{label}')
    axes[i].set_xticks([])
    axes[i].set_yticks([])
fig.subplots_adjust(top=1.4)
fig.suptitle(f'{n} Random Training Examples')
plt.show()
```

10 Random Training Examples



Shown below is the shape of the images. They are tensors of shape (1, 1, 28, 28). The first dimension is created by the `train_loader`, the next three dimensions are created by the `ToTensor` dataset transform.

- First dimension is the batch dimension. We set it to 1 in the `train_loader` above, but we will increase it when training.
- Second dimension is the channel dimension. Since MNIST images are grayscale they only have 1 channel (instead of the usual 3 channels for R G B).
- Third dimension is height. MNIST images are 28 pixels high.
- Fourth dimension is width. MNIST images are 28 pixels wide.

```
[4]: next(train_iter)[0].shape
```

```
[4]: torch.Size([1, 1, 28, 28])
```

Next we'll calculate the mean and standard deviation of the dataset so that we can normalize it properly before training.

```
[5]: def calculate_mean(loader: torch.utils.data.DataLoader) -> float:
    total_sum: float = 0.0
    total_pixels: int = 0

    x: Tensor
    for x, _ in loader:
        total_sum += x.sum()
        total_pixels += x.numel() # Alternatively can just do `+= 28*28`

    return (total_sum / total_pixels).item()

def calculate_std(loader: torch.utils.data.DataLoader, mean: float) -> float:
    squared_diff_sum: float = 0.0
    total_pixels: int = 0

    x: Tensor
    for x, _ in loader:
        squared_diff_sum += ((x - mean)**2).sum()
        total_pixels += x.numel() # Alternatively can just do `+= 28*28`
```

```
return (squared_diff_sum / total_pixels).sqrt().item()
```

```
[6]: x_mean = calculate_mean(train_loader)
      x_std = calculate_std(train_loader, x_mean)
      x_mean, x_std
```

```
[6]: (0.13066108524799347, 0.3081068992614746)
```

Now that we know the mean and standard deviation of the dataset, we can normalize it properly. After this transformation, the dataset should have a mean of 0 and standard deviation of 1.

```
[7]: transform = transforms.Compose([
      transforms.ToTensor(),                # (C, H, W)
      transforms.Normalize((x_mean,), (x_std,)) # (C, H, W)
  ])
```

```
[8]: train_dataset = datasets.MNIST(
      root=data_root, train=True, download=True, transform=transform
  )
  train_loader = torch.utils.data.DataLoader(
      train_dataset, batch_size=1, shuffle=True
  )
  normalized_x_mean = calculate_mean(train_loader)
  normalized_x_std = calculate_std(train_loader, normalized_x_mean)
  normalized_x_mean, normalized_x_std
```

```
[8]: (-1.9569970390875824e-06, 1.0000040531158447)
```

1.1 Training

```
[9]: import random

      import torch
      import torch.nn.functional as F

      import mnist
      from mnist.model.data_loader import DataLoaderScheduler
      from mnist.model.train import estimate_loss, train, train_basic_sdg
      from mnist.utils.plot import show_images_with_labels
      from mnist.utils.preprocessing import tensor2img
```

```
[10]: if torch.cuda.is_available():
        device = 'cuda'
      else:
        device = 'cpu'
      print(f'Using {device}')
```

Using cuda

We use training and validation datasets with a batch size of 64.

```
[11]: train_dataset = datasets.MNIST(root=data_root, train=True, download=True,
    ↪transform=transform)
    val_dataset = datasets.MNIST(root=data_root, train=False, download=True,
    ↪transform=transform)

    batch_size = 64

    train_loader = torch.utils.data.DataLoader(train_dataset,
    ↪batch_size=batch_size, shuffle=True)
    val_loader = torch.utils.data.DataLoader(val_dataset, batch_size=batch_size,
    ↪shuffle=True)
```

1.1.1 Small MLP

First we train with a simple one-layer multilayer perceptron. The layer has $28 \cdot 28 = 784$ inputs (one per pixel in the image) and 10 outputs (one for each class label).

```
[12]: small_mlp_layer_in_outs = [28*28, 10]
    mlp_small = mnist.model.mlp.MLP(small_mlp_layer_in_outs)
    mlp_small.to(device)
    diagnostics = train_basic_sdg(mlp_small, train_loader, val_loader, epochs=10,
    ↪max_lr=0.01, min_lr=0.001, flatten=True, device=device)
```

```
epoch: 0, lr: 1.0000e-02, train loss: 2.4267, train accuracy: 0.0702, val loss:
2.4482, val accuracy: 0.0698
epoch: 1, lr: 7.9433e-03, train loss: 0.3379, train accuracy: 0.9022, val loss:
0.3265, val accuracy: 0.9075
epoch: 2, lr: 6.3096e-03, train loss: 0.3172, train accuracy: 0.9100, val loss:
0.3098, val accuracy: 0.9127
epoch: 3, lr: 5.0119e-03, train loss: 0.3010, train accuracy: 0.9173, val loss:
0.3014, val accuracy: 0.9137
epoch: 4, lr: 3.9811e-03, train loss: 0.2959, train accuracy: 0.9134, val loss:
0.2949, val accuracy: 0.9194
epoch: 5, lr: 3.1623e-03, train loss: 0.2879, train accuracy: 0.9192, val loss:
0.2924, val accuracy: 0.9156
epoch: 6, lr: 2.5119e-03, train loss: 0.3057, train accuracy: 0.9133, val loss:
0.2899, val accuracy: 0.9195
epoch: 7, lr: 1.9953e-03, train loss: 0.2923, train accuracy: 0.9227, val loss:
0.2920, val accuracy: 0.9170
epoch: 8, lr: 1.5849e-03, train loss: 0.2683, train accuracy: 0.9236, val loss:
0.2793, val accuracy: 0.9202
epoch: 9, lr: 1.2589e-03, train loss: 0.2872, train accuracy: 0.9234, val loss:
0.2893, val accuracy: 0.9184
```

Observe in 10 training epochs we achieve:

```
[13]: small_mlp_last_epoch = diagnostics.loc[diagnostics['Epoch'].idxmax()]
      small_mlp_last_epoch
```

```
[13]: Epoch                9.000000
      Learning Rate        0.001259
      Training Loss        0.287175
      Training Accuracy     0.923438
      Validation Loss       0.289335
      Validation Accuracy   0.918438
      Name: 9, dtype: float64
```

1.1.2 Large MLP

Next, we train with a larger multilayer perceptron. There are five layers in this MLP starting with 784 inputs and finally reaching 10 outputs.

```
[14]: large_mlp_layer_in_outs = [28*28, 28*28*4, 28*28*16, 28*28*4, 10]
      mlp_large = mnist.model.mlp.MLP(large_mlp_layer_in_outs)
      mlp_large.to(device)
      diagnostics = train_basic_sdg(mlp_large, train_loader, val_loader, epochs=10,
      ↪max_lr=0.01, min_lr=0.001, flatten=True, device=device)
```

```
epoch: 0, lr: 1.0000e-02, train loss: 2.3311, train accuracy: 0.0634, val loss:
2.3333, val accuracy: 0.0605
epoch: 1, lr: 7.9433e-03, train loss: 0.3161, train accuracy: 0.9102, val loss:
0.2962, val accuracy: 0.9153
epoch: 2, lr: 6.3096e-03, train loss: 0.3073, train accuracy: 0.9108, val loss:
0.2828, val accuracy: 0.9161
epoch: 3, lr: 5.0119e-03, train loss: 0.2848, train accuracy: 0.9194, val loss:
0.2850, val accuracy: 0.9178
epoch: 4, lr: 3.9811e-03, train loss: 0.2657, train accuracy: 0.9269, val loss:
0.2772, val accuracy: 0.9191
epoch: 5, lr: 3.1623e-03, train loss: 0.2681, train accuracy: 0.9255, val loss:
0.2694, val accuracy: 0.9209
epoch: 6, lr: 2.5119e-03, train loss: 0.2443, train accuracy: 0.9306, val loss:
0.2617, val accuracy: 0.9266
epoch: 7, lr: 1.9953e-03, train loss: 0.2514, train accuracy: 0.9252, val loss:
0.2655, val accuracy: 0.9250
epoch: 8, lr: 1.5849e-03, train loss: 0.2825, train accuracy: 0.9242, val loss:
0.2840, val accuracy: 0.9206
epoch: 9, lr: 1.2589e-03, train loss: 0.2543, train accuracy: 0.9297, val loss:
0.2651, val accuracy: 0.9237
```

Observe in 10 training epochs we achieve:

```
[15]: large_mlp_last_epoch = diagnostics.loc[diagnostics['Epoch'].idxmax()]
      large_mlp_last_epoch
```

```
[15]: Epoch                9.000000
      Learning Rate        0.001259
      Training Loss        0.254267
      Training Accuracy    0.929688
      Validation Loss      0.265067
      Validation Accuracy   0.923750
      Name: 9, dtype: float64
```

Calculating the increase in synapses (excluding biases) from the small MLP to the large MLP and comparing it to the increase in accuracy shows that this architecture is not scaling well.

```
[16]: small_mlp_synapses = sum(n_in * n_out for n_in, n_out in
    ↪ zip(small_mlp_layer_in_outs, small_mlp_layer_in_outs[1:]))
large_mlp_synapses = sum(n_in * n_out for n_in, n_out in
    ↪ zip(large_mlp_layer_in_outs, large_mlp_layer_in_outs[1:]))
print(f'Synapses increased by a factor of {large_mlp_synapses:,}/
    ↪ {small_mlp_synapses:,} = {large_mlp_synapses / small_mlp_synapses:,.2f},
    ↪ validation accuracy increased by a factor of
    ↪ {large_mlp_last_epoch["Validation Accuracy"]}/
    ↪ {small_mlp_last_epoch["Validation Accuracy"]} =
    ↪ {large_mlp_last_epoch["Validation Accuracy"] /
    ↪ small_mlp_last_epoch["Validation Accuracy"]:,.4f}')
```

Synapses increased by a factor of $81,165,952 / 7,840 = 10,352.80$, validation accuracy increased by a factor of $0.92375 / 0.9184375 = 1.0058$

1.1.3 CNN

Next we try a convolutional neural network.

```
[17]: cnn = mnist.model.cnn.CNN()
      cnn.to(device)
      diagnostics = train_basic_sdg(cnn, train_loader, val_loader, epochs=10,
    ↪ max_lr=0.01, min_lr=0.001, flatten=False, device=device)
```

```
epoch: 0, lr: 1.0000e-02, train loss: 2.3268, train accuracy: 0.0780, val loss:
2.3296, val accuracy: 0.0689
epoch: 1, lr: 7.9433e-03, train loss: 0.0870, train accuracy: 0.9753, val loss:
0.0812, val accuracy: 0.9739
epoch: 2, lr: 6.3096e-03, train loss: 0.0660, train accuracy: 0.9784, val loss:
0.0612, val accuracy: 0.9803
epoch: 3, lr: 5.0119e-03, train loss: 0.0529, train accuracy: 0.9855, val loss:
0.0530, val accuracy: 0.9852
epoch: 4, lr: 3.9811e-03, train loss: 0.0468, train accuracy: 0.9881, val loss:
0.0509, val accuracy: 0.9834
epoch: 5, lr: 3.1623e-03, train loss: 0.0468, train accuracy: 0.9853, val loss:
0.0493, val accuracy: 0.9855
epoch: 6, lr: 2.5119e-03, train loss: 0.0410, train accuracy: 0.9897, val loss:
0.0435, val accuracy: 0.9866
```

```
epoch: 7, lr: 1.9953e-03, train loss: 0.0416, train accuracy: 0.9898, val loss:
0.0432, val accuracy: 0.9864
epoch: 8, lr: 1.5849e-03, train loss: 0.0352, train accuracy: 0.9902, val loss:
0.0425, val accuracy: 0.9861
epoch: 9, lr: 1.2589e-03, train loss: 0.0359, train accuracy: 0.9892, val loss:
0.0429, val accuracy: 0.9852
```

Observe in 10 training epochs we achieve:

```
[18]: cnn_last_epoch = diagnostics.loc[diagnostics['Epoch'].idxmax()]
      cnn_last_epoch
```

```
[18]: Epoch          9.000000
      Learning Rate    0.001259
      Training Loss    0.035926
      Training Accuracy 0.989219
      Validation Loss   0.042905
      Validation Accuracy 0.985156
      Name: 9, dtype: float64
```

The CNN is significantly more accurate than the large MLP while using significantly less parameters.

```
[19]: mlp_large_parameters = sum(p.numel() for p in mlp_large.parameters())
      cnn_parameters = sum(p.numel() for p in cnn.parameters())
      print(f'CNN has {cnn_parameters:,}/{mlp_large_parameters:,} = {100 *
      ↪cnn_parameters / mlp_large_parameters:.2f}% of the parameters of the large
      ↪MLP, yet it was {cnn_last_epoch["Validation Accuracy"]:.4f}/
      ↪{large_mlp_last_epoch["Validation Accuracy"]:.4f} =
      ↪{cnn_last_epoch["Validation Accuracy"]/large_mlp_last_epoch["Validation
      ↪Accuracy"]:.4f} times more accurate on the validation set')
```

CNN has 67,562/81,184,778 = 0.08% of the parameters of the large MLP, yet it was 0.9852/0.9237 = 1.0665 times more accurate on the validation set

1.1.4 Adding in Random Image Transformations to Improve Model Accuracy

Before we *really* train the CNN, we can apply some random transformations to the training dataset before running it through the network. Shown below are some training images and some random transformations applied on those images: - rotations between -30° and 30° - horizontal shifts between 0% and 25% of the image width and vertical shifts between 0% and 25% of the image height - scaling between 0.75 and 1.25 - shear between -25° and 25°

This will better reflect the variability of human handwriting.

```
[20]: transform1 = transforms.Compose([
      transforms.ToTensor(),                # (C, H, W)
      transforms.Normalize((x_mean,), (x_std,)) # (C, H, W)
  ])
      transform2 = transforms.Compose([
```

```

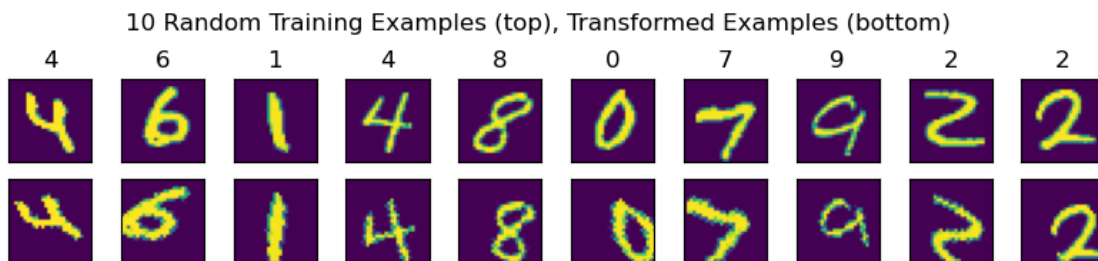
transforms.RandomAffine(
    degrees=30,
    translate=(0.25, 0.25),
    scale=(0.75, 1.25),
    shear=25
),
transforms.ToTensor(),                # (C, H, W)
transforms.Normalize((x_mean,), (x_std,)) # (C, H, W)
])

untransformed_train_dataset = datasets.MNIST(root=data_root, train=True,
↳download=True)

n = 10

fig, axes = plt.subplots(2, n, figsize=(n, 2))
for i in range(n):
    image, label = random.choice(untransformed_train_dataset)
    axes[0][i].imshow(tensor2img(transform1(image)).squeeze())
    axes[0][i].set_title(f'{label}')
    axes[0][i].set_xticks([])
    axes[0][i].set_yticks([])
    axes[1][i].imshow(tensor2img(transform2(image)).squeeze())
    axes[1][i].set_xticks([])
    axes[1][i].set_yticks([])
fig.subplots_adjust(top=0.75)
fig.suptitle(f'{n} Random Training Examples (top), Transformed Examples
↳(bottom)')
plt.show()

```



However, it is important to not introduce these transforms too early, as the model may overfit details early on that are not fundamentally part of the digit. We will start with a few epochs being trained on the un-transformed data then slowly ramp up the magnitude of the random transformations.

Conveniently for us, the `transforms.Compose` object is mutable, allowing us to change the parameters for the random transformations during training. This is what is implemented in our custom `DataLoaderScheduler` class.

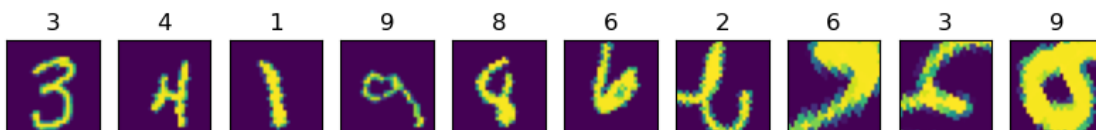

```
[21]: transform2 = transforms.Compose([
    transforms.RandomAffine(
        degrees=30,
        translate=(0.1, 0.1),
        scale=(0.9, 1.1),
        shear=5
    ),
    transforms.ToTensor(),
    transforms.Normalize((x_mean,), (x_std,)) # (C, H, W)
])

transformed_train_dataset = datasets.MNIST(root=data_root, train=True,
    ↳download=True, transform=transform2)
train_dataset_loader = torch.utils.data.DataLoader(transformed_train_dataset,
    ↳batch_size=1, shuffle=True)

n = 10

fig, axes = plt.subplots(1, n, figsize=(n, 2))
i = 0
for images, labels in train_dataset_loader:
    if i == n:
        break
    if i >= int(n/2):
        transform2.transforms[0].shear = [-30, 30]
        transform2.transforms[0].degrees = [90, 270]
        transform2.transforms[0].scale = [0.01, 2.9]
    image = images[0]
    label = labels[0]
    axes[i].imshow(tensor2img(image).squeeze())
    axes[i].set_title(label.item())
    axes[i].set_xticks([])
    axes[i].set_yticks([])
    i += 1
fig.subplots_adjust(top=0.75)
fig.suptitle(f'Note how the magnitude of the transformations increases halfway
    ↳through the batch')
plt.show()
```

Note how the magnitude of the transformations increases halfway through the batch



1.1.5 Actual Training Time

Here is the heavy-duty training setup for the convolutional neural network. We are using the data loader scheduler, which progressively increases the magnitude of the random transformations on the training data, along with AdamW for the optimizer and a cosine annealing learning rate scheduler, training for 100 epochs.

```
[22]: from pathlib import Path
import torch
import matplotlib.pyplot as plt
import mnist
from mnist.model.data_loader import StepDataLoaderScheduler
from mnist.model.train import train

epochs = 30
x_mean, x_std = (0.13066114485263824, 0.30810731649398804)
data_root = Path('../data')

if torch.cuda.is_available():
    device = 'cuda'
else:
    device = 'cpu'

print(f'Using {device}')

data_loader = StepDataLoaderScheduler(
    root=data_root,
    x_mean=x_mean,
    x_std=x_std,
    batch_size=64,
    epochs=epochs,
    degrees=5,
    translate=0.25,
    scale=0.25,
    shear=5,
    warmup_steps=1,
    step_size=10
)

cnn = mnist.model.cnn.CNN()
cnn.to(device)

optimizer = torch.optim.AdamW(cnn.parameters(), lr=1e-4, weight_decay=0.01)
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=epochs,
    ↪eta_min=1e-6)
```

```
diagnostics = train(cnn, data_loader, epochs, optimizer, scheduler,
    ↪flatten=False, device=device, quiet=True, show_progress=True)
```

Using cuda

100%| | 30/30 [03:03<00:00, 6.12s/it]

```
[23]: diagnostics.head()
```

```
[23]:
```

	Epoch	Learning Rate	Training Loss	Training Accuracy	Validation Loss	\
0	0	0.000100	2.295186	0.128281	2.298017	
1	1	0.000100	0.104081	0.973281	0.099340	
2	2	0.000099	0.058452	0.983750	0.062181	
3	3	0.000098	0.047779	0.985469	0.048903	
4	4	0.000096	0.047750	0.984531	0.056663	

```

Validation Accuracy
0      0.124844
1      0.972031
2      0.981719
3      0.983437
4      0.982344
```

```
[24]: diagnostics.tail()
```

```
[24]:
```

	Epoch	Learning Rate	Training Loss	Training Accuracy	Validation Loss	\
25	25	0.000008	0.002862	0.999844	0.042119	
26	26	0.000005	0.003427	0.999687	0.051729	
27	27	0.000003	0.003409	0.999531	0.054479	
28	28	0.000002	0.002719	0.999844	0.047883	
29	29	0.000001	0.002323	0.999844	0.048698	

```

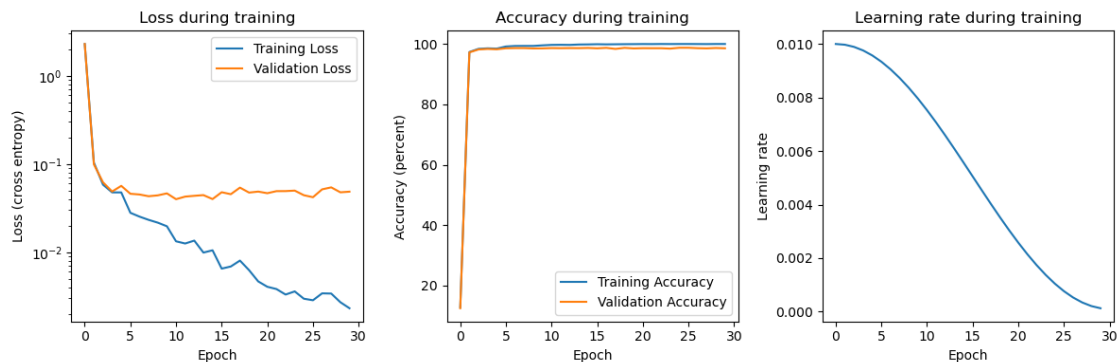
Validation Accuracy
25      0.987344
26      0.985938
27      0.985313
28      0.986563
29      0.985625
```

```
[25]: fix, axes = plt.subplots(1, 3, figsize=(12, 4))
axes[0].plot(diagnostics['Epoch'], diagnostics['Training Loss'],
    ↪label='Training Loss')
axes[0].plot(diagnostics['Epoch'], diagnostics['Validation Loss'],
    ↪label='Validation Loss')
axes[0].set_title('Loss during training')
axes[0].set_xlabel('Epoch')
axes[0].set_ylabel('Loss (cross entropy)')
axes[0].set_yscale('log')
```

```

axes[0].legend()
axes[1].plot(diagnostics['Epoch'], diagnostics['Training Accuracy'] * 100,
            label='Training Accuracy')
axes[1].plot(diagnostics['Epoch'], diagnostics['Validation Accuracy'] * 100,
            label='Validation Accuracy')
axes[1].set_title('Accuracy during training')
axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('Accuracy (percent)')
axes[1].legend()
axes[2].plot(diagnostics['Epoch'], diagnostics['Learning Rate'] * 100,
            label='Learning Rate')
axes[2].set_title('Learning rate during training')
axes[2].set_xlabel('Epoch')
axes[2].set_ylabel('Learning rate')
plt.tight_layout()
plt.show()

```



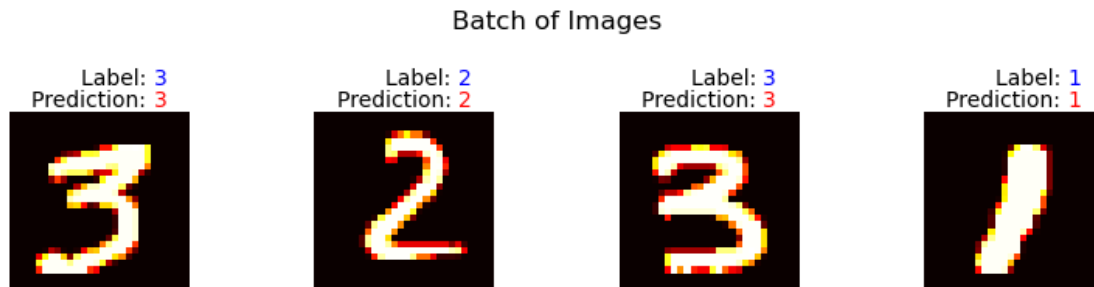
1.2 Evaluating the Model with Validation Data

```
[26]: val_iter = iter(data_loader.loader('val'))
```

```

[27]: from mnist.utils.plot import show_images_with_labels
classes = list(map(str, range(10)))
images, labels = next(val_iter)
images = images[:4].to(device)
labels = labels[:4].to(device)
output = cnn(images)
_, predicted = torch.max(output, 1)
show_images_with_labels(images.to('cpu'), classes, labels.to('cpu'), predicted.
                        to('cpu'))

```



Here we measure the loss and accuracy on the original validation set.

```
[28]: from mnist.model.train import dataset_loss
from torchvision import transforms, datasets
transform = transforms.Compose([
    transforms.ToTensor(),                # (C, H, W)
    transforms.Normalize((x_mean,), (x_std,)) # (C, H, W)
])
val_dataset = datasets.MNIST(root=data_root, train=False, download=True,
    ↪transform=transform)
batch_size = 64
val_loader = torch.utils.data.DataLoader(val_dataset, batch_size=batch_size,
    ↪shuffle=True)
estimated_loss = dataset_loss(cnn, val_loader, flatten=False, device=device)
print(f'Model achieved loss of {estimated_loss[0]:.4f} on the untransformed,
    ↪validation set (accuracy of {estimated_loss[1]*100:.2f}%).')
```

Model achieved loss of 0.0488 on the untransformed validation set (accuracy of 98.61%).

And here we measure the loss on the validation set with random affine transformations applied.

```
[29]: transform = transforms.Compose([
    transforms.RandomAffine(
        degrees=5,
        translate=(0.25, 0.25),
        scale=(0.75, 1.25),
        shear=5
    ),
    transforms.ToTensor(),                # (C, H, W)
    transforms.Normalize((x_mean,), (x_std,)) # (C, H, W)
])
val_dataset = datasets.MNIST(root=data_root, train=False, download=True,
    ↪transform=transform)
batch_size = 64
```

```

val_loader = torch.utils.data.DataLoader(val_dataset, batch_size=batch_size,
    ↪shuffle=True)
estimated_loss = dataset_loss(cnn, val_loader, flatten=False, device=device)
print(f'Model achieved loss of {estimated_loss[0]:.4f} on the transformed_
    ↪validation set (accuracy of {estimated_loss[1]*100:.2f}%).')

```

Model achieved loss of 8.7659 on the transformed validation set (accuracy of 35.53%).

We also have some hand-drawn digits that are not part of the MNIST dataset (I drew these myself :P). These images better reflect the kind of images that the model will receive in production, so it is important that it performs well on these.

```

[30]: from torchvision import transforms, datasets
test_data_root = Path('../custom-data')
x_mean, x_std = (0.13066114485263824, 0.30810731649398804)
transform = transforms.Compose([
    transforms.Grayscale(num_output_channels=1),
    transforms.ToTensor(),                    # (C, H, W)
    transforms.Normalize((x_mean,), (x_std,)) # (C, H, W)
])
test_dataset = datasets.ImageFolder(root=test_data_root, transform=transform)
test_dataloader = torch.utils.data.DataLoader(test_dataset, batch_size=1,
    ↪shuffle=True)

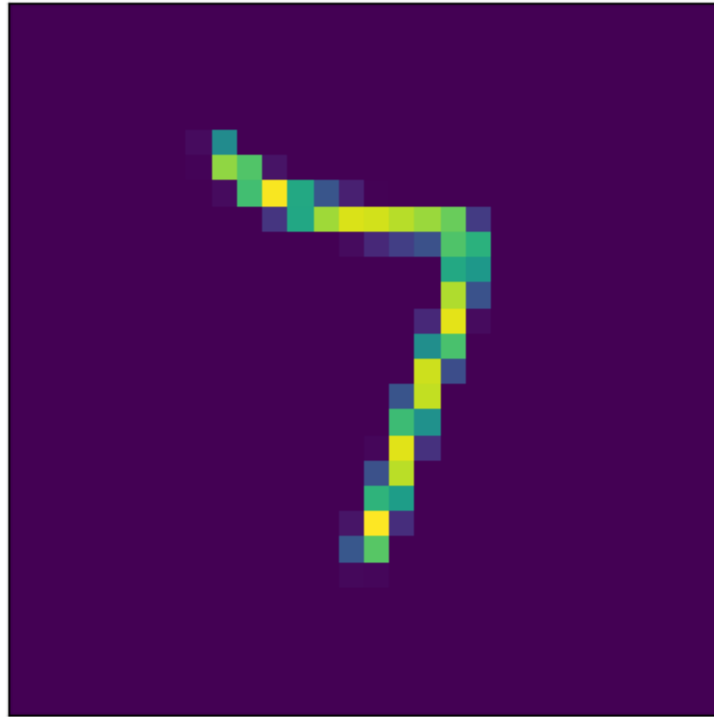
```

```

[31]: from mnist.utils.plot import tensor2img
images, labels = next(iter(test_dataloader))
plt.imshow(tensor2img(images[0]))
plt.title(str(labels[0].item()))
plt.xticks([])
plt.yticks([])
plt.show()

```

7



```
[32]: estimated_loss = dataset_loss(cnn, val_loader, flatten=False, device=device)
      print(f'Model achieved loss of {estimated_loss[0]:.4f} on the test set_
            ↳(accuracy of {estimated_loss[1]*100:.2f}%).')
```

Model achieved loss of 8.8160 on the test set (accuracy of 35.03%).

1.3 Saving the Model

Now we're going to save the model, both as a PyTorch state dict (so we can reload the model in our training setup), and as an ONNX model (which we will be using in production).

```
[33]: torch.save(cnn.state_dict(), './thousand-iteration-model.pth')
      print('Model saved')
```

Model saved

Here, we load the model from the state dict that we saved above and export it to ONNX.

```
[34]: cnn = mnist.model.cnn.CNN()
      state_dict = torch.load('thousand-iteration-model.pth')
      cnn.load_state_dict(state_dict)
      cnn.eval()
      dummy_input = torch.randn(1, 1, 28, 28)
      torch.onnx.export(
```

```

cnn,
dummy_input,
'model.onnx',
export_params=True,
opset_version=12,
do_constant_folding=True,
input_names=['input'],
output_names=['output'],
dynamic_axes={'input': {0: 'batch_size'}, 'output': {0: 'batch_size'}}
)

```

/tmp/ipykernel_59320/418881113.py:2: FutureWarning: You are using `torch.load` with `weights\_only=False` (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See <https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models> for more details). In a future release, the default value for `weights\_only` will be flipped to `True`. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via `torch.serialization.add\_safe\_globals`. We recommend you start setting `weights\_only=True` for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.

```
state_dict = torch.load('thousand-iteration-model.pth')
```

And here we load the ONNX model that we exported above and run inference on it.

```

[35]: import torch
import onnxruntime as ort
from torchvision import transforms, datasets
from mnist.utils.plot import show_images_with_labels
import numpy as np

x_mean, x_std = (0.13066114485263824, 0.30810731649398804)

transform = transforms.Compose([
    transforms.ToTensor(), # (C, H, W)
    transforms.Normalize((x_mean,), (x_std,)) # (C, H, W)
])

transformed_train_dataset = datasets.MNIST(root=data_root, train=True,
↳download=True, transform=transform)
train_dataset_loader = torch.utils.data.DataLoader(transformed_train_dataset,
↳batch_size=4, shuffle=True)

session = ort.InferenceSession('model.onnx')

```



```

inputs, labels = next(iter(train_dataset_loader))

outputs = session.run(None, {'input': inputs.numpy()})

predicted = np.argmax(outputs[0], axis=1) # Equivalent to `torch.max(outputs, 1)`

show_images_with_labels(inputs, list(map(str, range(10))), labels, predicted,
    ONNX)

```

ONNX

